UNIVERSIDADE CESUMAR – UNICESUMAR
ENGENHARIA DE SOFTWARE

Julia Alves Fungaes – RA: 26005177-2
Giovana Pinheiro Fertunani – RA: 26008421-2

PSE EM AÇÃO

Sistema de Planejamento e Acompanhamento de Ações de Saúde na Escola *Temática de contextualização: Alimentação Saudável*

AEP – 2ª Entrega: Implementação e Apresentação

Maringá – PR
2026
Julia Alves Fungaes
Giovana Pinheiro Fertunani

PSE EM AÇÃO

Sistema de Planejamento e Acompanhamento de Ações de Saúde na Escola

AEP — 2ª Entrega
(Implementação e Apresentação) — apresentada ao curso de Engenharia de Software da Universidade Cesumar – UniCesumar, turma ESOFT2NA, como requisito avaliativo referente ao 2º semestre de 2026.

Maringá – PR
2026

SUMÁRIO

1 ENGENHARIA DE SOFTWARE..... 4
1.1 Requisitos Funcionais (revisados)..... 4
1.2 Requisitos Não Funcionais (revisados)..... 4
1.3 Histórico de Alterações nos Requisitos 5
1.4 Diagrama de Casos de Uso 5
1.5 Descrição dos Casos de Uso Principais..... 6
2 ALGORITMOS E LÓGICA DE PROGRAMAÇÃO (REVISÃO) .....
8 2.1 Fluxograma Detalhado – Cadastrar Ação (revisado)..... 8
2.2 Pseudocódigo Revisado e Correspondência com a Implementação 9
3 IMPLEMENTAÇÃO EM LINGUAGEM C..... 10
3.1 Estrutura de Dados 10
3.2 Organização em Funções.....11 3.3
Tratamento de Situações Especiais.....11
4 TESTES REALIZADOS .....
11 5 MANUAL DO USUÁRIO.....
13 5.1 Objetivo 13 5.2
Requisitos para Utilização 13 5.3
Compilação e Execução.....13 5.4
Opções do Menu e Exemplos de Uso 14 5.5
Mensagens de Erro Mais Comuns.....20 5.6
Cuidados Éticos.....20 **6**
CONSIDERAÇÕES FINAIS 20
6.1 Resultados.....20 6.2
Reflexão Social 20 6.3
Limitações.....21 6.4
Aprendizados.....21 6.5
Possibilidades de Melhoria 21 **7**
REPOSITÓRIO E APRESENTAÇÃO EM VÍDEO 21
REFERÊNCIAS..... 22

1. ENGENHARIA DE SOFTWARE

1.1 Requisitos Funcionais (revisados)

A Tabela 1 apresenta os requisitos funcionais após a implementação. A coluna "Situação" indica se o requisito foi mantido, detalhado ou incluído em relação à 1ª entrega, e a seção 1.3 registra a justificativa de cada alteração.

Tabela 1 – Requisitos funcionais

ID	Descrição	Situação	Caso de uso

RF1	O sistema deve permitir cadastrar uma ação do PSE informando código, escola, tema, data prevista, público-alvo, responsável, quantidade prevista de participantes; a situação inicial é “Planejada”.	Detalhado	UC01
RF2	O sistema deve listar todas as ações cadastradas de forma organizada, em formato de tabela, exibindo os principais dados de cada registro.	Mantido	UC02
RF3	O sistema deve permitir pesquisar ações por código, escola ou tema (escola e tema são comparados pela escrita exata, como foram cadastrados).	Detalhado	UC03
RF4	O sistema deve permitir atualizar a situação de uma ação para planejada, realizada ou cancelada.	Mantido	UC04
RF5	O sistema deve registrar a quantidade efetiva de participantes quando a ação for marcada como realizada.	Desdobrado do RF4	UC05
RF6	O sistema deve gerar um resumo geral com a quantidade de ações planejadas, realizadas e canceladas, o total de participantes e o percentual de participação em relação ao número previsto das ações realizadas.	Detalhado	UC06
RF7	O sistema deve validar entradas críticas, impedindo códigos repetidos ou não positivos, quantidades negativas ou nulas, campos de texto vazios, datas fora do formato DD/MM/AAAA e opções inexistentes no menu.	Detalhado	UC08
RF8	O sistema deve informar o limite de armazenamento (10 ações) e impedir novos cadastros quando ele for atingido.	Novo	UC01
RF9	O sistema deve exibir informações sobre o sistema, incluindo seus limites éticos (uso de dados fictícios e coletivos, sem diagnóstico).	Novo	UC07

1.2 Requisitos Não Funcionais (revisados)

Tabela 2 – Requisitos não funcionais

ID	Descrição	Situação
RNF 1	O sistema deve ser executado em ambiente de terminal (linha de comando), sem dependência de interface gráfica, compilado com gcc, utilizando somente os recursos básicos de C vistos em aula.	Detalhado
RNF 2	O sistema deve apresentar mensagens claras de confirmação de operações e de erro, garantindo usabilidade mesmo para usuários sem experiência técnica.	Mantido
RNF 3	O código-fonte deve ser modularizado em funções com responsabilidades bem definidas, evitando concentrar a lógica na função main.	Mantido

ID	Descrição	Situação
RNF 4	O sistema não deve armazenar nome, diagnóstico, prontuário, condição clínica ou qualquer dado sensível individual de estudantes; somente dados fictícios e coletivos.	Novo

RNF 5	O programa não deve encerrar de forma inesperada diante de entradas inválidas (como letras no lugar de números), tratando-as com mensagem e nova solicitação.	Novo

1.3 Histórico de Alterações nos Requisitos

Conforme previsto na 1ª entrega, os requisitos foram revisados durante a implementação. As mudanças abaixo foram motivadas por situações encontradas na codificação e nos testes:

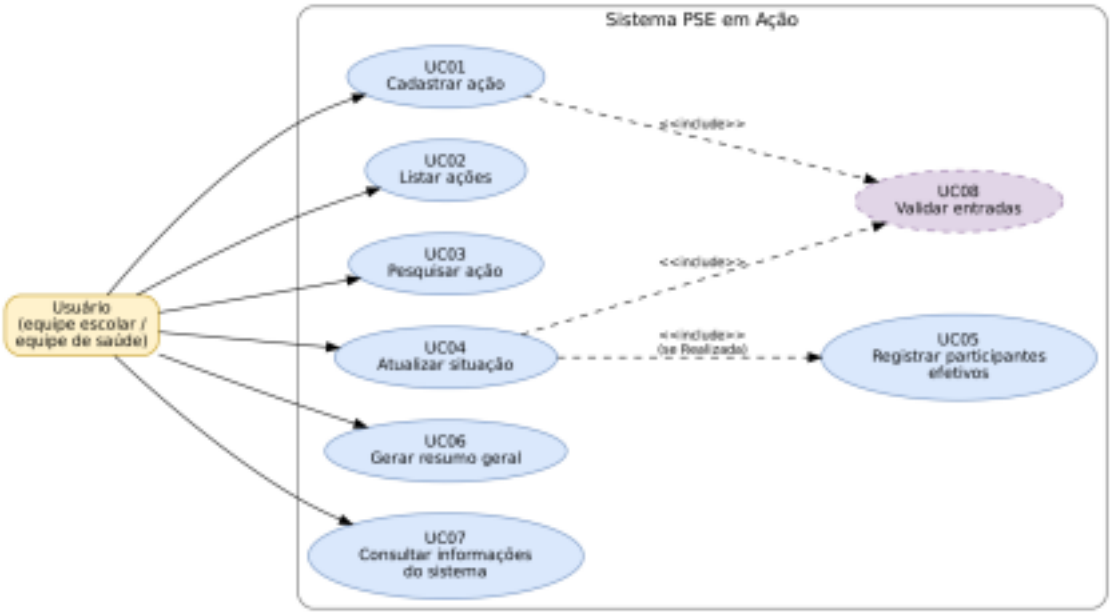
- **RF4 → RF4 e RF5:** o registro de participantes efetivos foi separado da troca de situação por ser uma regra própria, que só se aplica às ações realizadas e que gera um caso de uso incluído (UC05).
- **RF1 e RF7 (quantidade prevista):** a validação passou de "não negativa" para "maior que zero", pois uma ação planejada com zero participantes previstos tornaria o percentual de participação sem sentido.
- **RF7 (data):** foi incluída a validação do formato DD/MM/AAAA (10 caracteres, com barras nas posições corretas), porque o campo de data é um texto e aceitaria qualquer valor. A validação do calendário (dias de cada mês) ficou como melhoria futura.
- **RF6 (percentual):** o percentual passou a ser calculado sobre o número previsto das ações **realizadas**; incluir ações planejadas ou canceladas reduziria artificialmente o índice. Sem ações realizadas, o sistema informa que o percentual está indisponível, evitando divisão por zero.
- **RF3 e RF1 (textos):** para manter o código no nível do conteúdo estudado, os textos são lidos com scanf("%s"), sem espaços (usa-se _ no lugar), e a pesquisa por escola e tema compara a escrita exata com strcmp(), em vez de busca parcial.
- **RF8 e RF9:** incluídos para tratar o limite de armazenamento exigido pela 2ª entrega e para tornar visíveis os limites éticos do sistema.
- **RNF4 e RNF5:** incluídos para formalizar a restrição de privacidade do desafio e a robustez diante de entradas inválidas.
- **Atualização de situação:** ao voltar uma ação para "Planejada" ou ao marcá-la como "Cancelada", os participantes efetivos são zerados, mantendo a consistência do resumo.

1.4 Diagrama de Casos de Uso

O diagrama representa o único ator do sistema — o usuário (membro da equipe escolar ou de saúde) — e as funcionalidades implementadas. A validação de entradas (UC08) é

incluída pelas operações que recebem dados, e o registro de participantes efetivos (UC05) é incluído pela atualização de situação quando a ação é marcada como realizada.

Figura 1 – Diagrama de casos de uso do sistema PSE em Ação



Fonte: elaborada pelas autoras (2026).

1.5 Descrição dos Casos de Uso Principais

UC01 – Cadastrar ação	
Objetivo	Registrar uma nova ação coletiva do PSE no sistema.
Ator	Usuário (equipe escolar ou de saúde).
Pré-condições	O programa está em execução e o menu principal está exibido; há espaço disponível no vetor (menos de 10 ações).
Fluxo principal	1. O usuário escolhe a opção 1 do menu. 2. O sistema solicita o código, a escola, o tema, a data prevista, o público-alvo, o responsável e a quantidade prevista de participantes (textos sem espaços, usando _). 3. O usuário informa os dados, que são validados (UC08) a cada campo. 4. O sistema grava a ação com situação “Planejada” e participantes efetivos igual a 0. 5. O sistema exibe a mensagem de sucesso e retorna ao menu.
Fluxos alternativos	A1. Limite de armazenamento atingido: o sistema informa o limite e retorna ao menu sem solicitar dados. A2. Código inválido (não numérico, zero ou negativo) ou duplicado: o sistema informa o erro e retorna ao menu. A3. Data fora do formato DD/MM/AAAA ou quantidade menor ou igual a zero: o sistema exibe o erro e solicita novamente o mesmo campo.
Pós-condições	A nova ação passa a constar da lista e pode ser listada, pesquisada, atualizada e contabilizada no resumo.

UC02 – Listar ações

Objetivo	Apresentar todas as ações cadastradas de forma organizada.
Ator	Usuário.
Pré-condições	O menu principal está exibido.
Fluxo principal	1. O usuário escolhe a opção 2. 2. O sistema exibe, para cada ação, o código, a escola, o tema, a data, os participantes previstos e efetivos e a situação. 3. O sistema retorna ao menu.
Fluxos alternativos	A1. Nenhuma ação cadastrada: o sistema informa que não há ações e retorna ao menu.
Pós-condições	Nenhum dado é alterado.

UC03 – Pesquisar ação	
Objetivo	Localizar rapidamente ações por código, escola ou tema.
Ator	Usuário.
Pré-condições	Existe ao menos uma ação cadastrada.
Fluxo principal	1. O usuário escolhe a opção 3 e o critério (1-código, 2-escola, 3-tema). 2. O sistema solicita o valor a pesquisar. 3. O sistema percorre o vetor e exibe os detalhes completos das ações cujo valor é igual ao informado (nas pesquisas por escola e tema, a escrita deve ser igual à cadastrada). 4. O sistema retorna ao menu.
Fluxos alternativos	A1. Nenhuma ação cadastrada: o sistema informa e retorna ao menu. A2. Critério inexistente: o sistema exibe mensagem de erro e retorna ao menu. A3. Registro inexistente: o sistema informa que nenhuma ação foi encontrada.
Pós-condições	Nenhum dado é alterado.

UC04 – Atualizar situação (inclui UC05 – Registrar participantes efetivos)	
Objetivo	Alterar a situação de uma ação para planejada, realizada ou cancelada e, quando realizada, registrar a quantidade efetiva de participantes.
Ator	Usuário.
Pré-condições	Existe ao menos uma ação cadastrada.
Fluxo principal	1. O usuário escolhe a opção 4 e informa o código da ação. 2. O sistema exibe a situação atual e as opções (1-Planejada, 2-Realizada, 3- Cancelada). 3. O usuário escolhe “Realizada”. 4. (UC05) O sistema solicita a quantidade efetiva de participantes, valida que não é negativa e a registra. 5. O sistema atualiza a situação, confirma a operação e retorna ao menu.

Fluxos alternativos	A1. Código inexistente: o sistema informa o erro e retorna ao menu. A2. Quantidade efetiva negativa: o sistema exibe o erro e solicita novamente.

	A3. Situação “Planejada” ou “Cancelada”: não há solicitação de participantes e a quantidade efetiva é zerada. A4. Situação inexistente: o sistema informa o erro e não altera o registro.
Pós-condições	A situação (e, se realizada, a quantidade efetiva) da ação fica atualizada e é refletida na listagem e no resumo.

UC06 – Gerar resumo geral	
Objetivo	Consolidar as informações gerais de acompanhamento das ações.
Ator	Usuário.
Pré-condições	Existe ao menos uma ação cadastrada.
Fluxo principal	1. O usuário escolhe a opção 5. 2. O sistema conta as ações por situação, soma os participantes efetivos das ações realizadas e calcula o percentual em relação ao número previsto dessas ações. 3. O sistema exibe o resumo e retorna ao menu.
Fluxos alternativos	A1. Nenhuma ação cadastrada: o sistema informa e retorna ao menu. A2. Nenhuma ação realizada: o percentual é exibido como indisponível.
Pós-condições	Nenhum dado é alterado.

UC07 – Consultar informações do sistema	
Objetivo	Exibir a finalidade do sistema, sua capacidade e seus limites éticos.
Ator	Usuário.
Pré-condições	O menu principal está exibido.
Fluxo principal	1. O usuário escolhe a opção 6. 2. O sistema exibe as informações e retorna ao menu.
Fluxos alternativos	Não há.
Pós-condições	Nenhum dado é alterado.

O caso de uso **UC08 – Validar entradas** não é acionado diretamente pelo usuário: ele é incluído por UC01 e UC04 e, de forma geral, por todas as leituras do menu e dos campos numéricos (relação <<include>> no diagrama).

2. ALGORITMOS E LÓGICA DE PROGRAMAÇÃO (REVISÃO) Os fluxogramas geral e do cadastro e os pseudocódigos da 1ª entrega foram mantidos. Esta seção apresenta apenas o que mudou após a implementação.

2.1 Fluxograma Detalhado – Cadastrar Ação (revisado)

Durante a codificação, o fluxo foi simplificado ao conteúdo estudado em aula. Código inválido (menor ou igual a zero) ou duplicado encerra o cadastro com uma mensagem de erro;

já a data (formato DD/MM/AAAA) e a quantidade prevista (maior que zero) são lidas novamente até ficarem corretas. Os textos são lidos com scanf("%s"), que sempre exige ao menos uma palavra, e por isso os campos de texto não ficam vazios. A Figura 2 representa o fluxo efetivamente implementado na função cadastrarAcao().

Figura 2 – Fluxograma revisado do cadastro de ação (versão implementada)



Fonte: elaborada pelas autoras (2026).

2.2 Pseudocódigo Revisado e Correspondência com a Implementação O pseudocódigo abaixo, em Portugol, corresponde à função de cadastro implementada em C.

```
funcao cadastrarAcao()
{
 se(total >= MAX)
 { escreva("Limite de armazenamento atingido.") retorne }
```



```
 leia(codigo)
 se(codigo <= 0) { escreva("Código inválido.") retorne }
 se(existeCodigo(codigo)) { escreva("Código duplicado.") retorne }
leia(escola, tema)
 leia(data)
 enquanto(formato da data for inválido) { escreva("Data inválida.") leia(data) }
leia(publico, responsavel)
 leia(quantidade)
 enquanto(quantidade <= 0) { escreva("Quantidade inválida.") leia(quantidade) }
acoes[total] = nova acao (situacao = 1 Planejada, efetiva = 0)  total = total + 1
 escreva("Ação cadastrada com sucesso!")
```

```
}
```

A Tabela 3 relaciona cada operação do fluxograma geral à função correspondente no código-fonte, evidenciando a coerência entre planejamento e implementação.

Tabela 3 – Correspondência entre o fluxograma geral e as funções em C

Opção	Operação	Função em C	Observação
1	Cadastrar ação	cadastrarAcao()	Valida limite, código, data e quantidade
2	Listar ações	listarAcoes()	Exibe tabela organizada com todas as ações
3	Pesquisar ação	pesquisarAcao()	Busca por código, escola ou tema (valor exato)
4	Atualizar situação	atualizarSituacao()	Registra participantes efetivos quando realizada
5	Gerar resumo geral	gerarResumo()	Totais por situação, participantes e percentual
6	Informações do sistema	mostrarInformacoes()	Exibe limites éticos e capacidade
7	Sair	main() – laço do-while	Encerra o laço do menu

Fonte: elaborada pelas autoras (2026).

3. IMPLEMENTAÇÃO EM LINGUAGEM C

3.1 Estrutura de Dados

Cada ação é representada por uma struct chamada Acao, e o conjunto das ações é guardado em um vetor global com capacidade para MAX (10) elementos, acompanhado de um contador do total cadastrado. A situação é guardada como número inteiro: 1 (Planejada), 2 (Realizada) e 3 (Cancelada).

```
struct Acao {
 int codigo;
 char escola[50];
 char tema[50];
 char data[11]; /* DD/MM/AAAA */
 char publico[50];
 char responsavel[50];
 int qtdPrevista;
 int qtdEfetiva;
 int situacao; /* 1, 2 ou 3 */
```



```
};

struct Acao acoes[MAX];
int total = 0;
```

3.2 Organização em Funções

Tabela 4 – Funções do programa e suas responsabilidades

Função	Responsabilidade
main()	Controla o laço do menu principal e chama a função de cada opção (switch).
mostrarMenu()	Exibe as opções do menu.
lerInteiro()	Lê um número inteiro e pede novamente se o usuário digitar algo que não seja número.
buscarPorCodigo()	Procura uma ação pelo código e devolve sua posição no vetor (ou -1).
imprimirSituacao()	Escreve o nome da situação (Planejada, Realizada ou Cancelada).
mostrarAcao()	Mostra todos os dados de uma ação.
cadastrarAcao()	Implementa o cadastro com as validações (RF1, RF7, RF8).
listarAcoes()	Lista todas as ações cadastradas (RF2).
pesquisarAcao()	Pesquisa por código, escola ou tema (RF3).
atualizarSituacao()	Atualiza a situação e registra participantes efetivos (RF4, RF5).
gerarResumo()	Calcula e exibe o resumo geral (RF6).
mostrarInformacoes()	Exibe informações e limites éticos (RF9).

3.3 Tratamento de Situações Especiais

- **Registro inexistente:** pesquisa e atualização por código informam que nenhuma ação foi encontrada, sem alterar dados.
- **Código duplicado:** o cadastro é interrompido com mensagem de erro, mantendo a unicidade dos códigos.
- **Limite de armazenamento atingido:** ao tentar cadastrar a 11ª ação, o sistema informa o limite de 10 ações e retorna ao menu.
- **Entradas não numéricas:** a função lerInteiro() confere o retorno do scanf e pede o número novamente, evitando que o programa trave ao digitar letras.
- **Textos:** são digitados sem espaços (usa-se _), pois são lidos com scanf("%s").

O código-fonte completo está disponível no repositório GitHub do projeto.

4. TESTES REALIZADOS

Os testes foram executados manualmente no terminal, na versão final do programa, compilada com gcc 13.3.0 (Ubuntu 24.04) usando as opções -Wall -Wextra, sem nenhum aviso

de compilação. Cada caso foi executado simulando a digitação do usuário, e o resultado obtido foi comparado ao esperado.

Tabela 5 – Registro dos casos de teste

ID	Cenário	Entrada	Resultado esperado	Resultado
CT01	Cadastro valido de uma ação	Todos os campos validos	Mensagem de sucesso e situação inicial “Planejada”.	Aprovado
CT02	Código duplicado	Código 101 cadastrado duas vezes	Erro informando que já existe ação com esse código.	Aprovado
CT03	Código nao numerico	Código 'abc' e depois 0	Erro de entrada não numérica; depois, erro de código menor ou igual a zero.	Aprovado
CT04	Código negativo	Código -5	Erro informando que o código deve ser maior que zero.	Aprovado
CT05	Data em formato inválido	Data 31-02-2026 e depois 10/10/2026	Erro de data inválida e nova solicitação; após a data correta, cadastro concluído.	Aprovado
CT06	Quantidade prevista negativa ou zero	Quantidades -4, 0 e depois 25	Erro de quantidade; após informar 25, cadastro concluído.	Aprovado
CT07	Listagem com sistema vazio	Opção 2 sem registros	Mensagem de que nenhuma ação está cadastrada.	Aprovado
CT08	Listagem com ações cadastradas	Duas ações cadastradas	As duas ações cadastradas são exibidas.	Aprovado
CT09	Pesquisa por código existente	Código 102	Detalhes completos da ação 102.	Aprovado
CT10	Pesquisa por código inexistente	Código 999	Mensagem de nenhuma ação encontrada com esse código.	Aprovado
CT11	Pesquisa por escola	Escola_Monteiro_Lobato	As duas ações da escola (101 e 103) são exibidas.	Aprovado
CT12	Pesquisa por escola inexistente	Escola_Z	Mensagem de nenhuma ação encontrada para essa escola.	Aprovado
CT13	Pesquisa por tema	Atividade_física	A ação 102 é exibida.	Aprovado
CT14	Critério de pesquisa inexistente	Critério 9	Erro de opção de pesquisa inexistente.	Aprovado

CT15	Atualizar para realizada com participantes	Código 101, situação 2, 35 participantes	Situação atualizada com sucesso.	Aprovado
CT16	Participantes efetivos negativos	-1 e depois 20	Erro de quantidade negativa; após 20, atualização concluída.	Aprovado
CT17	Atualizar ação inexistente	Código 555	Erro informando que a ação não foi encontrada.	Aprovado
CT18	Situação inexistente na atualização	Situação 8	Erro de situação inexistente, sem alterar o registro.	Aprovado

ID	Cenário	Entrada	Resultado esperado	Resultado
CT19	Resumo geral com percentual	3 ações: 1 realizada (35/40), 1 cancelada, 1 planejada	1 planejada, 1 realizada, 1 cancelada; 35 participantes; 87,5%.	Aprovado
CT20	Resumo sem ações	Opção 5 sem registros	Mensagem de que nenhuma ação está cadastrada.	Aprovado
CT21	Limite de armazenamento	11º cadastro (limite = 10)	Erro de limite de armazenamento atingido.	Aprovado
CT22	Opção inexistente no menu	Opções 9 e 'abc'	Mensagem de opção inválida e de entrada não numérica.	Aprovado
CT23	Encerramento do programa	Opção 7	Mensagem de encerramento e fim do programa.	Aprovado

Dos 23 casos executados, 23 foram aprovados. Os testes cobrem as seis funcionalidades mínimas e as validações exigidas (registro inexistente, código duplicado e limite de armazenamento).

5. MANUAL DO USUÁRIO

5.1 Objetivo

O PSE em Ação é um programa de terminal que ajuda equipes escolares e de saúde a planejar, registrar e acompanhar ações coletivas do Programa Saúde na Escola, como oficinas de alimentação saudável, apresentando informações claras sobre situação e participação. O programa utiliza apenas dados fictícios e coletivos e não registra informações individuais de estudantes.

5.2 Requisitos para Utilização

• Compilador C (gcc ou compatível) — em Windows, por exemplo, MinGW-w64 ou MSYS2; em Linux e macOS, o gcc/clang já costuma estar disponível.

• Um terminal (Prompt de Comando, PowerShell, Terminal do Linux ou macOS).

5.3 Compilação e Execução

Com o repositório baixado, abra o terminal na pasta do projeto e execute:

```
# Compilar
gcc -Wall -Wextra -o pse src/main.c

# Executar no Linux/macOS
./pse

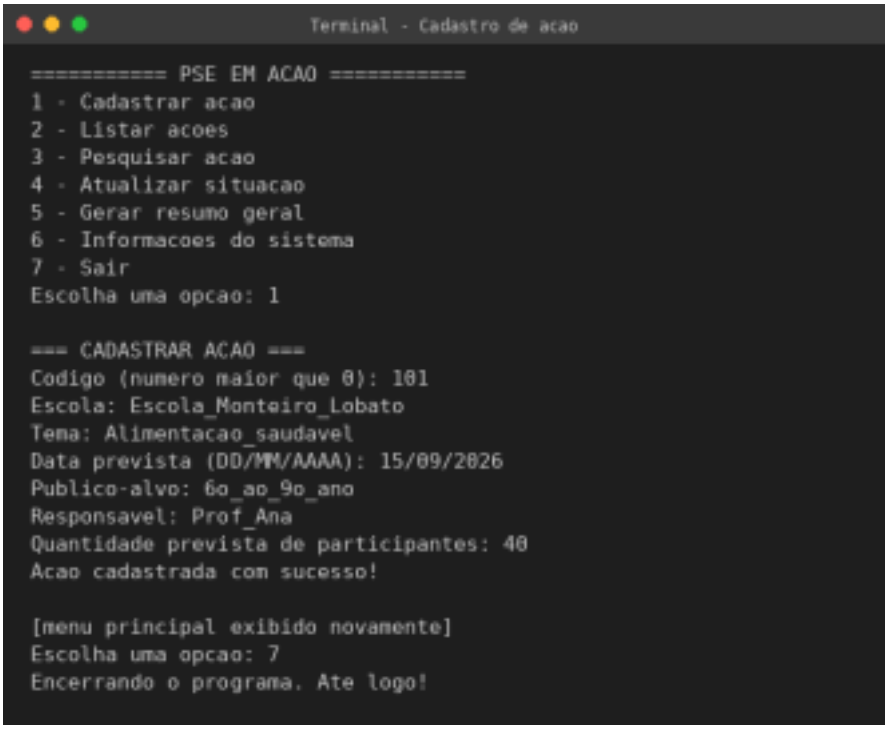
# Executar no Windows
pse.exe
```


Os dados ficam apenas na memória: ao encerrar o programa (opção 7), todos os registros são descartados. O programa comporta até 10 ações por execução.

5.4 Opções do Menu e Exemplos de Uso

Ao iniciar, o programa exibe o menu principal e aguarda a digitação de uma opção (1 a 7), seguida de Enter. Após cada operação, o menu é exibido novamente até que o usuário escolha a opção 7.

Figura 3 – Cadastro de ação (opção 1) com dados fictícios



Fonte: elaborada pelas autoras (2026).

Opção 1 – Cadastrar ação. Informe, na ordem solicitada: código (número inteiro

positivo e único), escola, tema, data prevista (DD/MM/AAAA), público-alvo, responsável e quantidade prevista de participantes (maior que zero). **Os textos devem ser digitados sem espaços, usando _ no lugar (ex.: Escola_Monteiro_Lobato).** Se a data ou a quantidade forem inválidas, o programa informa o erro e pede novamente; código inválido ou repetido cancela o cadastro (Figura 4). A ação nasce com a situação “Planejada”.

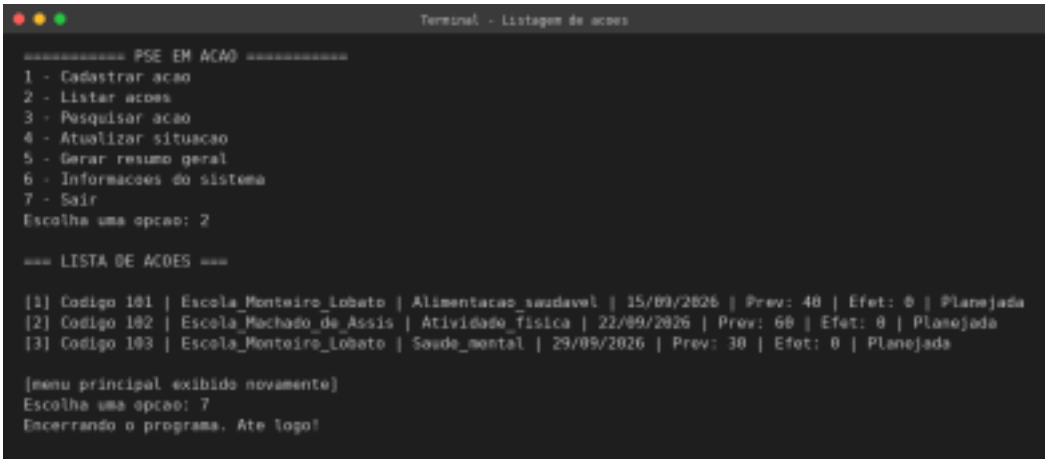
Figura 4 – Validações de entrada: código duplicado, data inválida e quantidade negativa



Fonte: elaborada pelas autoras (2026).

Opção 2 – Listar ações. Exibe, em uma linha por ação, o código, a escola, o tema, a data, os participantes previstos e efetivos e a situação (Figura 5).

Figura 5 – Listagem das ações cadastradas (opção 2)



Fonte: elaborada pelas autoras (2026).

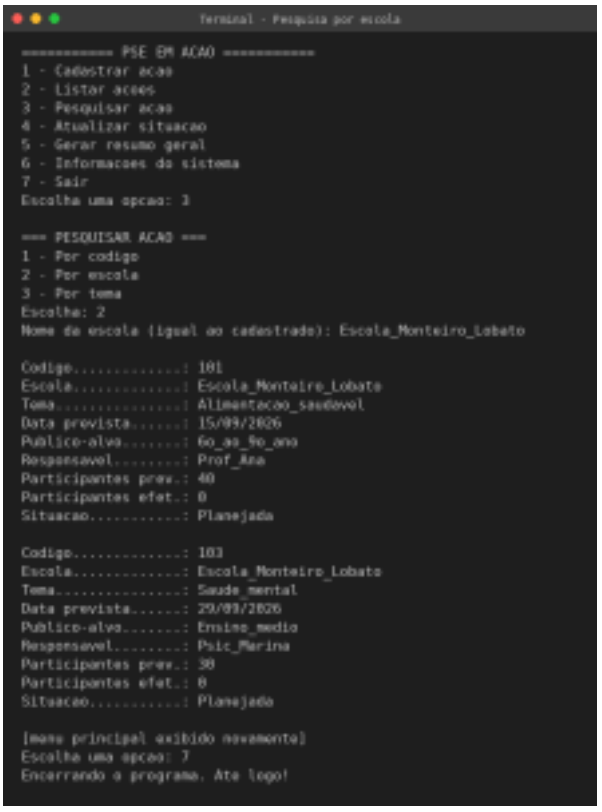
Opção 3 – Pesquisar ação. Escolha o critério: 1 (código, que mostra os detalhes completos da ação — Figura 6), 2 (escola) ou 3 (tema). Nos dois últimos, digite o nome exatamente como foi cadastrado, com maiúsculas e minúsculas e com _ (Figura 7).

Figura 6 – Pesquisa por código (opção 3, critério 1)



Fonte: elaborada pelas autoras (2026).

Figura 7 – Pesquisa por escola (opção 3, critério 2)



Fonte: elaborada pelas autoras (2026).

Opção 4 – Atualizar situação. Informe o código da ação e escolha a nova situação (1-Planejada, 2-Realizada, 3-Cancelada). Ao escolher “Realizada”, informe a quantidade efetiva de participantes (zero ou mais). Veja a Figura 8.

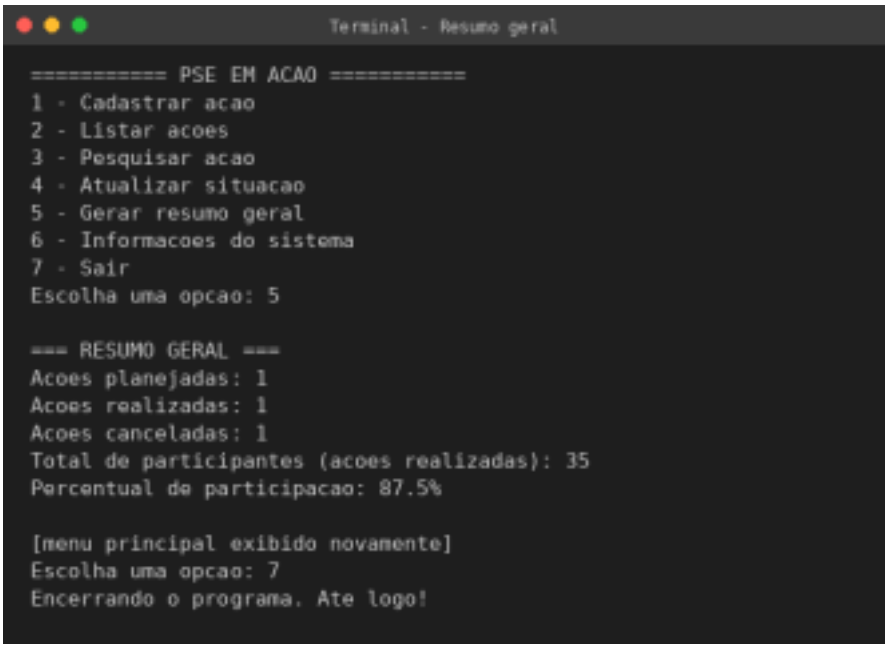
Figura 8 – Atualização de situação com validação de participantes (opção 4)



Fonte: elaborada pelas autoras (2026).

Opção 5 – Gerar resumo geral. Mostra o total de ações por situação, o total de participantes das ações realizadas e o percentual de participação em relação ao número previsto dessas ações (Figura 9).

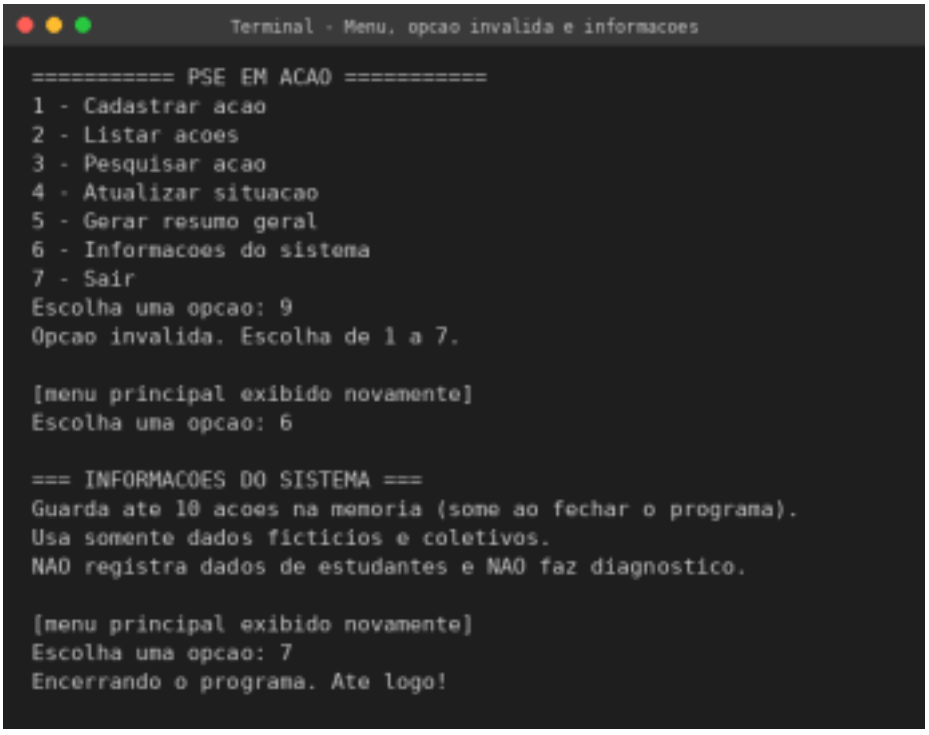
Figura 9 – Resumo geral (opção 5)



Fonte: elaborada pelas autoras (2026).

Opção 6 – Informações do sistema. Exibe a finalidade, a capacidade e os limites éticos do programa. **Opção 7 – Sair.** Encerra o programa. Digitar uma opção inexistente ou um texto não numérico gera uma mensagem de erro e o menu é exibido novamente (Figura 10).

Figura 10 – Opção inválida e informações do sistema (opção 6)



Fonte: elaborada pelas autoras (2026).

5.5 Mensagens de Erro Mais Comuns

Tabela 6 – Mensagens de erro e como resolver

Mensagem	O que fazer
“já existe uma ação com esse código”	Escolha outro código; cada ação deve ter código único.
“data inválida. Use o formato DD/MM/AAAA”	Digite a data com dois dígitos para dia e mês e quatro para o ano (ex.: 15/09/2026).
“digite apenas números”	Em campos numéricos, digite somente algarismos, sem letras ou símbolos.
“o código deve ser maior que zero”	Informe um código positivo (1, 2, 3...).
“limite de armazenamento atingido”	O programa comporta 10 ações por execução; reinicie para um novo conjunto.
“nenhuma ação encontrada”	Confira o código, a escola ou o tema na listagem (opção 2), lembrando de digitar igual ao cadastrado.

5.6 Cuidados Éticos

Utilize somente dados fictícios e informações coletivas. Não digite nomes de estudantes, diagnósticos, condições clínicas ou outros dados sensíveis nos campos do programa. O sistema não realiza diagnóstico, triagem, prescrição ou recomendação de tratamento; ele apenas organiza informações de planejamento.

6. CONSIDERAÇÕES FINAIS

6.1 Resultados

O sistema desenvolvido atende ao escopo mínimo do desafio: cadastra, lista, pesquisa, atualiza a situação, registra participantes efetivos e gera o resumo geral das ações do PSE, com menu controlado por estrutura de repetição, modularização em funções, uso de struct e vetor e validação de entradas. Os 23 casos de teste executados foram aprovados, e o código compila sem avisos com as opções -Wall -Wextra. A estrutura geral não depende da temática: embora a demonstração utilize principalmente a alimentação saudável, o campo “tema” permite cadastrar ações de qualquer eixo do PSE, como atividade física, saúde mental e saúde bucal.

6.2 Reflexão Social

O sistema é voltado a profissionais da equipe pedagógica, da Atenção Primária à Saúde e à gestão intersetorial. As principais dificuldades de uso previstas são a necessidade de familiaridade mínima com o terminal e a perda dos dados ao encerrar o programa. Em contrapartida, as informações consolidadas — situação das ações, participação e cancelamentos — podem apoiar o planejamento, ao mostrar quais escolas e temas têm menor alcance e onde é

preciso reforçar a continuidade das iniciativas, enfrentando a fragilidade de ações pontuais apontada por Sousa, Esperidião e Medina (2017). Do ponto de vista ético, o projeto adota privacidade desde o desenho: não há campo para dados individuais, o uso é restrito a dados fictícios e coletivos, e o sistema não diagnostica nem recomenda tratamentos.

6.3 Limitações

- Armazenamento somente em memória: os dados são perdidos ao encerrar o programa.
- Capacidade fixa de 10 ações por execução, definida em tempo de compilação. • Interface restrita ao terminal e sem edição dos demais campos de uma ação (apenas a situação e os participantes efetivos).
- Sem controle de acesso, histórico de alterações ou exclusão de ações. • A data é validada apenas pelo formato, sem conferir o calendário (por exemplo, 31/02/2026 é aceita) nem compará-la à data atual.
- Os textos não aceitam espaços (usa-se _) e a pesquisa por escola e tema exige o nome exato.

6.4 Aprendizados

O projeto permitiu praticar, de forma integrada, a análise de um problema social real, a modelagem com fluxogramas e casos de uso, a definição de requisitos e a implementação em C. Entre os principais aprendizados estão a importância de modularizar o código em funções pequenas, o tratamento de entradas inválidas no scanf, a necessidade de testar cada validação e a revisão dos requisitos à medida que a implementação revela situações não previstas (como o cálculo do percentual sem ações realizadas).

6.5 Possibilidades de Melhoria

- Leitura de textos com espaços (fgets) e pesquisa parcial por escola e tema (strstr), além de validação completa do calendário na data.
- Persistência em arquivo (texto ou binário), para manter os registros entre execuções.
- Alocação dinâmica de memória, eliminando o limite fixo de ações.
- Edição e exclusão de ações e ordenação da listagem por data ou escola. • Resumo por escola e por tema, e exportação do relatório em arquivo. • Comparação da data prevista com a data atual e alertas de ações atrasadas.

REFERÊNCIAS

SOUSA, Marta Caires de; ESPERIDIÃO, Monique Azevedo; MEDINA, Maria Guadalupe. A intersetorialidade no Programa Saúde na Escola: avaliação do processo político-gerencial e das práticas de trabalho. **Ciência & Saúde Coletiva**, Rio de Janeiro, v. 22, n. 6, p. 1781-1790, 2017. DOI: 10.1590/1413-81232017226.24262016.